

DATASET

Step 1: Collection of Benign Dataset

Before simulating malware attacks, it's essential to have a baseline dataset consisting of benign activities. This dataset typically represents normal operations without any malicious intent. Here's how this step might be carried out:

- 1. Data Selection:** Identify and gather a diverse set of benign activities or processes. This could include common user applications, system processes, and other legitimate software.
- 2. Data Gathering:** Collect data while these benign processes are running. This data can include various HPC events.

Step 2: Simulating Malware Attacks

Now, the focus shifts to simulating malware attacks on top of the benign dataset. This step involves executing malware programs and observing their behavior under different conditions:

- 1. Program Execution:** Run the malware programs on different CPU cores. This step involves parallelizing the execution of malware to observe how they interact with system resources when executed concurrently.
- 2. Timing Attacks:** Introduce time-based attacks by triggering malware at specific time points during the runtime of benign processes. This step aims to capture the effect of malware interference at different stages of normal system operation.
- 3. Data Collection:** Record various metrics during the execution of malware attacks, similar to the benign dataset collection. This includes HPC events. Additionally, note the timestamp of each attack to correlate it with the runtime of benign processes.

Step 3: Dataset Expansion

To create a diverse and comprehensive dataset, it's crucial to explore various permutations of core allocation and malware execution timings. This step involves systematically varying these parameters to capture a wide range of scenarios:

- 1. Core Allocation:** Execute malware on different CPU cores or combinations of cores. This helps understand how malware behavior differs based on resource allocation.
- 2. Malware Timing:** Vary the timing of malware attacks within the runtime of benign processes. This includes early attacks, late attacks, and attacks at intermediate stages.

3. Dataset Size: Collect sufficient data to ensure statistical significance. The dataset size mentioned (1 GB) indicates a substantial amount of data collection to cover diverse scenarios adequately which will be truncated.

Overview

In this approach we divide our malware detection task into two stages. The first stage is a continuous prediction model, that predicts the HPC event values based on 10 prior values. Then, it compares these predictions with the actual values and flags the region in the actual data that appear to be suspicious. This is done by training the model on benign datasets. The comparison between predicted values and original values is made using cosine similarity score, and a score less than a threshold of 0.3 is flagged. For inference, data prior and post to this flag checkpoint is saved and sent to stage 2.

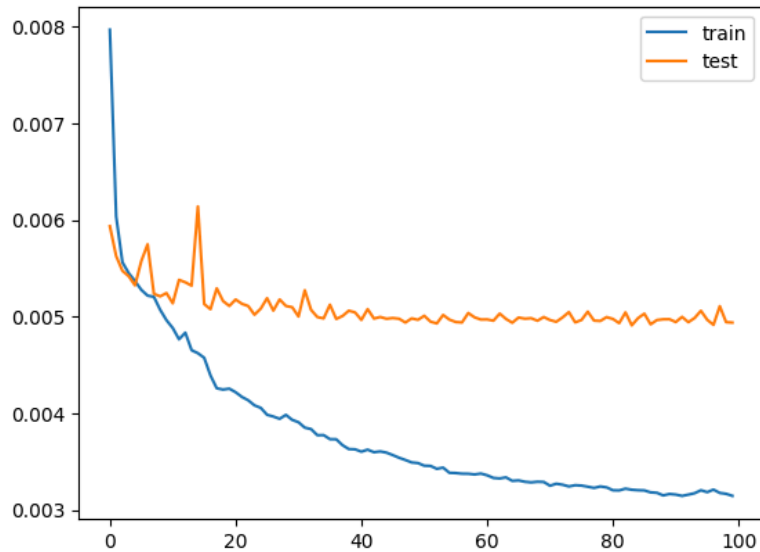
In the stage 2, we infer 2 things from the flagged dataset : whether the flagged window is malicious or not, and if yes, on which core is the malware running. This is done using a 5 class classification method.

The reason to do this to save some time for the users. The main malware prediction only happens in the flagged region, rather than at all times.

Stage 1:

- First step is to create a sequential time series model.
- This model takes a series of HPC Event data as input, and predicts the next HPC Events data based on the input.
- The window size is of 10.
- The training and test loss is shown as below.

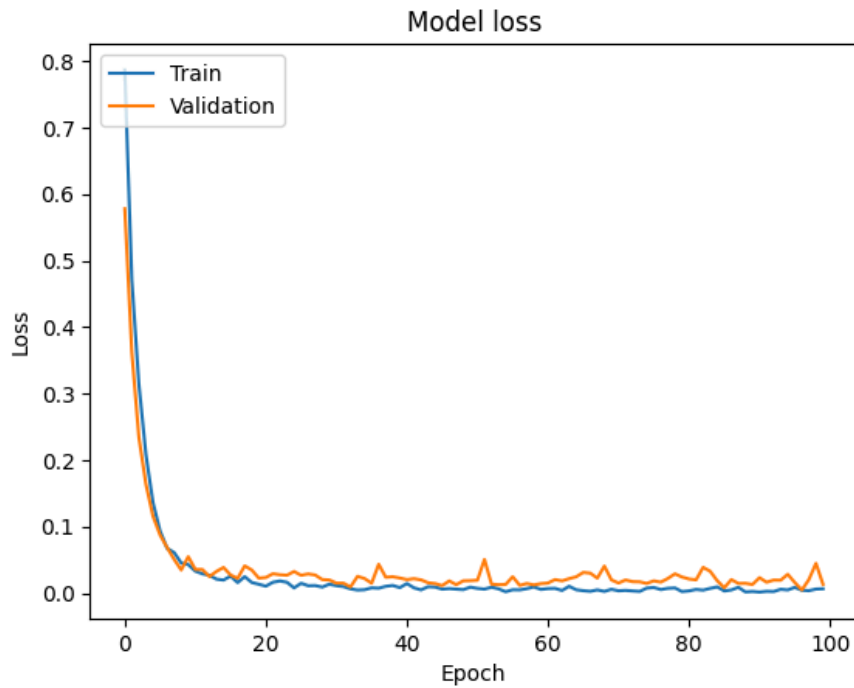
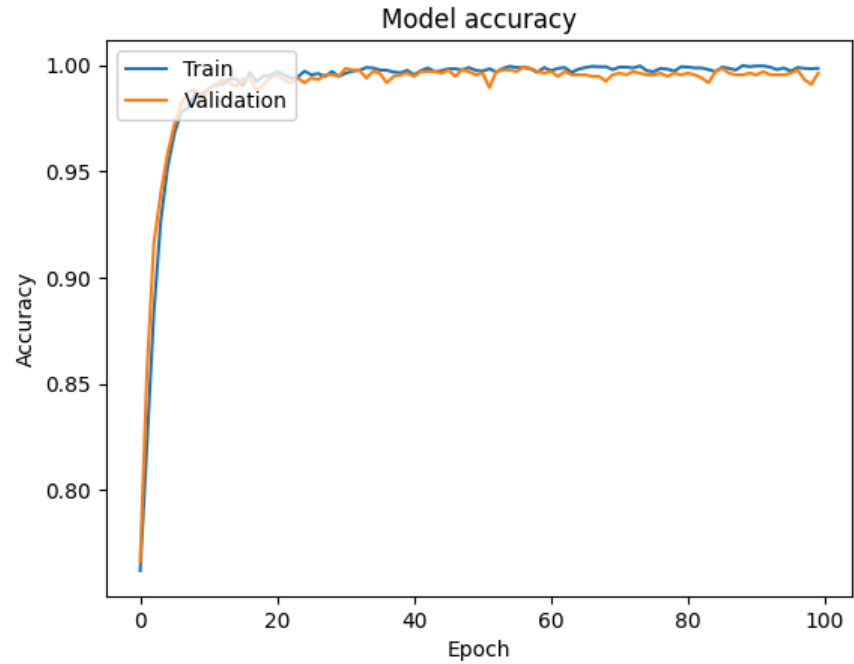
```
model = Sequential([
    LSTM(100, input_shape=(train_X.shape[1], train_X.shape[2]),
return_sequences=True),
    Dropout(0.2),
    LSTM(50),
    Dropout(0.2),
])
model.add(Dense(X.shape[2]))
opt = keras.optimizers.Adam(learning_rate=0.01)
```



Stage 2:

- This is another sequential model used for a classification task.
- Here, we predict the presence of a malware, and if present the core of the malware.
- The train and test loss and accuracy are as follows:

```
num_classes = 5
•
•
model = Sequential([
    LSTM(128, input_shape=(X_train.shape[1], X_train.shape[2]),
        return_sequences=True),
    Dropout(0.2),
    LSTM(32),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dense(num_classes, activation='softmax')
])
optimizer = Adam(lr=0.001)
model.compile(optimizer=optimizer,
    loss='categorical_crossentropy',
    metrics=['accuracy'])
```

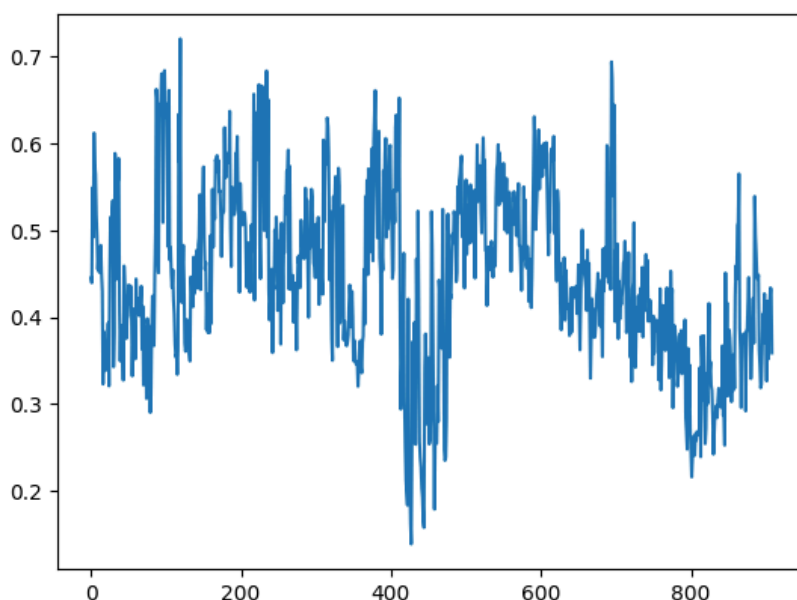


Inference Stage:

For inference, we have used

“merge_sort211309_m1=1_m2=_m3=_m4=_t1=11309__t2=467_t3=996_t4=119_n1=_n2=_n3=_n4=_20240319_181640_processed.csv” dataset, which has malware on core 2(0 based indexing, converted to core 3, where 0 means no malware), at timestamp = t1 = 11309.

On this file, from stage one we get the following cosine similarity plot between the predicted and true values:



This plot clearly shows that something fishy is going in the range 400 to 500, we flag that and send it to stage 2.

Predictions on stage 2, however dont seem very promising:

	precision	recall	f1-score	support
0	0.07	1.00	0.13	7
2	0.00	0.00	0.00	0
3	1.00	0.02	0.03	133
accuracy			0.06	140
macro avg	0.36	0.34	0.05	140
weighted avg	0.95	0.06	0.03	140